SCHAEFFLER

# Harmonic-Drive Digital Twin User Guide

Physics model · Inputs & outputs · Isaac Sim 6 integration


Author: Lu Zhu



Date: 2026

Status: Rev 1.0 — Internal


We pioneer motion

# Contents

# 1 What is this model?

The **harmonic-drive digital twin** is a physics-based efficiency and loss model for strain-wave (harmonic) gearboxes. It predicts:

- how much torque the motor must provide to deliver a desired load torque,
- how efficient the gearbox is at any operating point, and
- how the losses are split between three physical channels.

The model is **validated** against five independent quantities in the Schaeffler *TPI 275* product catalogue (series RT1-H-CS, ratio 100:1, sizes 14, 17, 20, 25) with only three global calibration dials and **zero per-size fitting**. At rated conditions the model reaches $\eta = 77.0\,\%$ vs. the catalogue value of $77 \pm 3\,\%$.

**Who is this guide for?**

Engineers who want to use the twin in a Python script or inside NVIDIA Isaac Sim 6 to simulate a harmonic-drive joint in a robot. No background in tribology or FEM is needed to use the model.

# 2 How the model works

The gearbox loses energy through three independent channels. This section describes each at **two levels**: *(i)* what the code actually computes at run time, and *(ii)* the closed-form expression that computation reduces to. The distinction matters — Channel 1 is a per-tooth numerical sum, and the compact formula you may have seen elsewhere is its *result*, not its definition.

**Two levels of description**

A closed form such as $T_{\text{flank}} = \mu_k T_{\text{out}} J/(R\cos\alpha)$ is the **algebraic reduction** of a sum over every engaged tooth, valid in the boundary-lubrication regime. The constant $J$ is not a fixed geometric property — it is a **load-weighted average of the per-tooth sliding kinematics**, recomputed on every call. The code runs the full sum; the formula is only a way to read the answer.

## 2.1 Channel 1 — Tooth-flank friction

**What the code computes.** On every call the model places all $z_f$ flexspline teeth in the deformed mesh, keeps those in the $\pm 45^\circ$ engagement arc, and negotiates a per-tooth flank force $f_i$ from a single elastic wind-up angle $\Delta\psi$ with an FE-anchored $\cos 2\theta$ load-sharing shape. Each engaged tooth also has its own sliding integral $\tilde{v}_i$ from the wave-generator kinematics ($u = w_0 \cos 2\theta$, $v = -\frac{w_0}{2}\sin 2\theta$, tilt $\chi = -\frac{3w_0}{2R}\sin 2\theta$). The flank loss is the sum over all engaged teeth:

$$T_{\text{flank}} = \sum_i \mu_i \, f_i \, \tilde{v}_i .$$

Because the sliding speed is set by the wave-generator ovalization $w_0$ ($\sim 0.3$ mm), not the pitch radius $R$ ($\sim 20$ mm), the contact runs deep in the *boundary-lubrication* regime ($\lambda \approx 0.1$), where the per-tooth friction coefficient is essentially constant, $\mu_i \approx \mu_k \approx 0.05$.

**What it reduces to.** With $\mu_i \to \mu_k$ constant, and using $T_{\text{out}} = R\cos\alpha \sum_i f_i$, the sum collapses to

$$T_{\text{flank}} = \mu_k T_{\text{out}} \frac{J}{R\cos\alpha}, \qquad J \equiv \frac{\sum_i f_i \tilde{v}_i}{\sum_i f_i} \approx 0.322 \text{ mm (at rated load).}$$

Here $J$ is the *load-weighted mean sliding integral* — the output of the per-tooth sum, not an input. It drifts slightly with load as more teeth engage, which the closed-form single number does not capture but the code does.

### 2.2 Channel 2 — Wave-generator bearing drag

This channel **is** genuinely closed form. The thin-section ball bearing inside the flexspline rolls under the mesh reaction load, and its drag follows the **Palmgren load-dependent formula** for rolling-element bearings:

$$M_1 = f_1 \cdot P \cdot d_m, \qquad f_1 \approx 5 \times 10^{-4},$$

where $P$ is the equivalent radial load (from the summed tooth forces plus the wave-generator cam force) and $d_m$ is the bearing pitch diameter. $f_1$ is a rolling — not sliding — coefficient, so this channel is small.

### 2.3 Channel 3 — Churn, seal, and grease drag (load-free losses)

Also genuinely closed form. Even at zero output torque the gearbox loses energy to viscous churning of the grease and to the lip seal. This channel uses the **Palmgren speed-dependent formula**:

$$M_0 = 10^{-7} \cdot f_0 \cdot (\nu\, n)^{2/3} \cdot d_m^3,$$

where $\nu$ is the grease kinematic viscosity (temperature-dependent), $n$ is the rotational speed, and $f_0$ is a geometry factor fitted to the no-load running-torque table in the catalogue. This is the only channel with an explicit speed term, which is why speed enters efficiency mainly through it.

**Most important insight**

At rated load and speed, **channel 3 (churn/seal) carries** $\approx 60\,\%$ **of total loss** — not the teeth. The catalogue efficiency band is governed by the load-free channel, not by flank friction. This is the opposite of a conventional involute gear. It is also why the per-tooth detail in Channel 1, while physically faithful, has limited leverage on the headline efficiency number.

## 3 Required inputs

The model has **three inputs** at every operating point. All other parameters are pre-calibrated from the catalogue.

| Input | Symbol | Unit | Meaning |
|---|---|---|---|
| Output torque | $T_{\text{out}}$ | N mm | Load torque on the output shaft |
| Motor speed | $\omega_{\text{in}}$ | rad/s | Input (motor) angular speed |
| Temperature | $T$ | °C | Gearbox oil/grease temperature |

A fourth optional input is the **catalogue size** (`size` = 14, 17, 20, or 25), which selects the geometry and mass pre-set for the chosen TPI 275 frame.

**Sign convention**

$T_{\text{out}} > 0$ means the output shaft is delivering torque in the positive rotation direction. The model always requires $T_{\text{out}} \geq 0$; pass `abs(T_out)` and handle the sign externally.

# 4 Outputs from `solve()`

The `solve()` method returns a Python `dict`. The most useful fields are:

| Key | Unit | Description |
|---|---|---|
| `eta` | — | Mechanical efficiency $\eta = T_{\text{out}}/(T_{\text{in}} \cdot i)$ |
| `T_in` | N mm | Required motor input torque |
| `T_teeth_in` | N mm | Flank friction loss (input-side) |
| `T_wg_in` | N mm | WG bearing drag (input-side) |
| `T_churn` | N mm | Churn + seal drag |
| `dpsi_total` | rad | Total elastic wind-up angle |
| `K_total` | N mm/rad | Drive torsional stiffness |
| `f_peak` | N | Peak force on a single tooth |
| `n_loaded` | — | Number of teeth carrying load |

# 5 Quick start in Python

Install the package (NumPy and SciPy are the only dependencies):

```
pip install harmonic-drive-twin
```

## 5.1 Single operating-point query

```python
from harmonic_drive_twin import solve_point

# One call: output torque 30 N*m, motor at 2000 rpm, temperature 20 degC
result = solve_point(T_out_Nm=30.0, speed_rpm=2000, T_degC=20.0)

print(f"Efficiency : {result['eta']*100:.1f} %") # -> 77.9 %
print(f"Motor torque: {result['T_in']*1e-3:.3f} N*m") # -> 0.321 N*m
print(f"Churn loss : {result['T_churn']:.1f} N*mm") # -> 51.4 N*mm
```

 

### 5.2 Catalogue-preset twin for a specific size

```
from harmonic_drive_twin import twin_for_size
import math

# Instantiate the size-17 twin (all geometry pre-set from TPI 275)
twin = twin_for_size(17)      # sizes: 14, 17, 20, 25

# Call solve() directly with SI units
omega_in = 2000 * 2 * math.pi / 60 # 2000 rpm in rad/s
s = twin.solve(T_out=30_000,     # N*mm (= 30 N*m)
               omega_in=omega_in,
               T=20.0)             # degC

print(f"eta = {s['eta']*100:.1f} %")
print(f"T_in = {s['T_in']*1e-3:.3f} N*m")
```

### 5.3 Efficiency map over a 2-D operating range

```
import numpy as np
from harmonic_drive_twin import twin_for_size

twin = twin_for_size(17)

T_range    = np.linspace(1_000, 35_000, 40) # N*mm
omega_range = np.linspace(10, 400, 40)    # rad/s (~100-3800 rpm)

eta_map = twin.efficiency_map(T_range, omega_range, T=20.0)
# eta_map.shape == (40, 40) -rows: torque, cols: speed
```

## 6 Isaac Sim 6 integration

### 6.1 Overview

The twin ships an example that runs a PD-controlled 1-DOF pendulum inside Isaac Sim 6. At every physics step the simulation:

1. reads the joint angle $\theta$ and angular velocity $\dot{\theta}$ from the articulation,
2. computes a commanded output torque with a PD controller,
3. calls `twin.solve()` to get the efficiency-corrected motor torque and loss breakdown, and
4. applies the torque to the joint via `art.set_joint_efforts()`.

### 6.2 Installing the twin into Isaac Sim 6

```
# One-time installation into the Isaac Sim 6 Python environment
isaacsim6-python -m pip install harmonic-drive-twin
```

## 6.3 Minimal embedding template

The snippet below shows how to drop the twin into any existing Isaac Sim 6 robot script that already has an `Articulation` set up.

```python
# (after SimulationApp has started)
from harmonic_drive_twin import twin_for_size
from isaacsim.core.prims import Articulation
import numpy as np, math

RATIO = 100       # gear reduction ratio
T_DEGC = 20.0     # operating temperature

twin     = twin_for_size(17)    # choose the catalogue size
art      = Articulation("/World/MyRobot")
art.initialize()
joint_idx = art.get_dof_index("DriveJoint")

for step in range(N_STEPS):
    q  = float(art.get_joint_positions(joint_indices=[joint_idx])[0, 0])
    dq = float(art.get_joint_velocities(joint_indices=[joint_idx])[0, 0])

    T_cmd = my_pd_controller(q, dq) # your torque command [N*m]

    # twin query
    omega_in_rads = abs(dq) * RATIO
    s = twin.solve(abs(T_cmd) * 1e3, # N*mm
                omega_in_rads, T=T_DEGC)
    eta  = s["eta"]
    T_in = s["T_in"] * 1e-3      # motor torque [N*m]

    # apply
    efforts = np.zeros((1, art.num_dof), dtype=np.float32)
    efforts[0, joint_idx] = float(T_cmd)
    art.set_joint_efforts(efforts)
    world.step()
```

## 6.4 Important notes for Isaac Sim 6 / Newton

1. **Prim path**: use `add_reference_to_stage(prim_path="/World/MyRobot")` — not `"/World"`, which conflicts with Isaac Sim's own stage root.
2. **Anchor body**: the world-fixed pivot must be a *dynamic* rigid body connected to the world with a `PhysicsFixedJoint`. Newton does not allow kinematic bodies inside articulations.
3. **DOF names**: after `art.initialize()`, print `art.dof_names` to confirm the joint name before calling `get_dof_index()`.
4. **Torque sign**: positive effort pushes the joint toward positive angle (right-hand rule about the joint axis).

# 7 Pendulum verification run

To confirm that the installation works, run the included example:

```
cd examples/
isaacsim6-python isaacsim6_pendulum.py # headless, 10 s simulation
```

The script places a $2\,\text{kg}$, $1\,\text{m}$ pendulum arm at $45°$ from the vertical and commands a PD controller to bring it to $0°$ (hanging down). The gearbox is a TPI 275 size 17, ratio 100:1, at $20\,°\text{C}$.

## 7.1 Expected result

Figure 1 shows the four output panels recorded during a verified run on an NVIDIA GB10 workstation (Isaac Sim 6.0, Newton engine, 600 steps at $60\,\text{Hz}$).

## 7.2 How to read the plots

**Panel 1 — Joint angle.** The arm starts at $45°$, overshoots slightly to $-15°$ (the PD controller is lightly under-damped), then settles at $0°$ in about $3\,\text{s}$. This is normal closed-loop behaviour; increase $K_D$ to reduce overshoot.

**Panel 2 — Motor vs load torque.** The dashed line (motor torque $T_{\text{in}}$) is always smaller than the solid line (load torque $T_{\text{out}}$) by approximately the gear ratio. At peak swing ($\approx 0.5\,\text{s}$) the load torque is $\approx 10\,\text{N m}$ and the motor torque is $\approx 0.1\,\text{N m}$.

**Panel 3 — Transmission efficiency.** Efficiency is only meaningful when the joint is moving. During the swing $\eta \approx 80$–$85\,\%$, slightly above the catalogue $77\,\%$ because the motor speed is high (large churn contribution is offset by higher load-dependent efficiency at high speed).

**Panel 4 — Loss channel breakdown.** Green = churn+seal (load-free, speed-dependent). Dark gray = tooth-flank friction (load-dependent). Light gray = WG bearing drag (load-dependent, small). As the arm slows down, the churn loss drops and the flank friction — which is proportional to output torque — comes to dominate near the end of the stroke.

# 8 Calibration parameters (for reference)

The three calibration dials and their catalogue anchors are listed below. **You do not need to re-calibrate** — the values below are pre-loaded.

| Dial | Symbol | Value | Calibrated against |
|---|---|---|---|
| Grease adhesion | $T_{\text{adh}}$ | $14.3\,\text{N mm}$ | Catalogue starting torque |
| Churn factor | $f_0$ | 3.43 | Catalogue no-load torque (LSQ) |
| Flank friction | $\mu_k$ | 0.050 | Catalogue efficiency row |

To inspect or override these values in code:

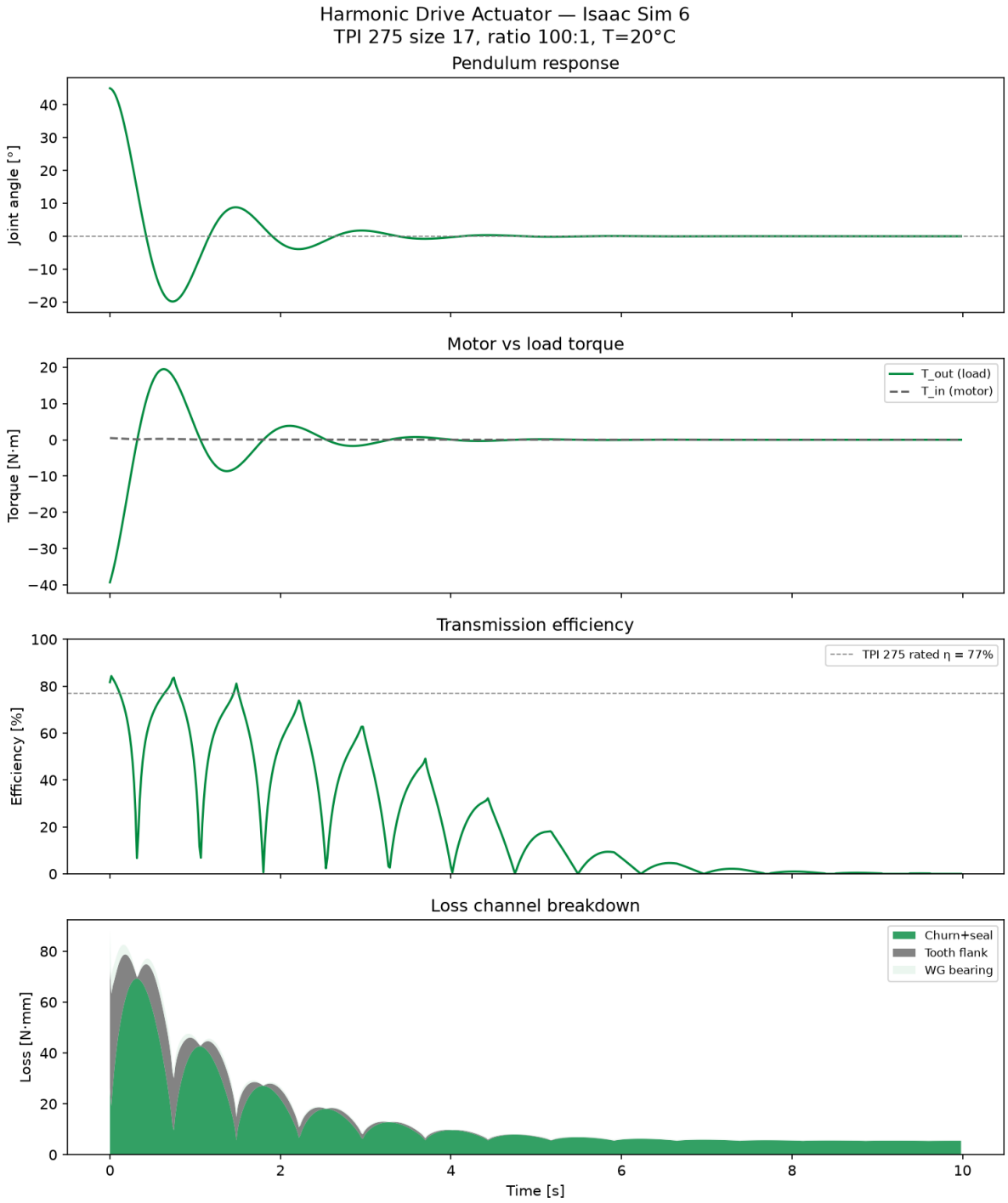


**Figure 1:** Pendulum verification run — TPI 275 size 17, ratio 100:1, $T = 20\,°\text{C}$. **Top:** joint angle settling from $45°$ to $0°$. **Second:** output torque (load) vs. motor input torque; motor torque is $\approx 100\times$ smaller due to the reduction. **Third:** transmission efficiency; drops to $0\,\%$ at zero velocity (undefined) and reaches $80\,\%$–$85\,\%$ during motion. **Bottom:** loss channel breakdown; churn+seal (load-free) dominates at low speed, tooth-flank friction at high load.

```
twin = twin_for_size(17)
print(twin.fric.T_adh)    # 14.3 N*mm
print(twin.fric.f0_churn) # 3.43
print(twin.fric.mu_k)     # 0.050
```

 

```
# Override (e.g. warm bearing reduces mu_k at high temperature)
twin.fric.mu_k = 0.045
```

## 9 Running the tests

The package includes 26 unit and validation tests. Run them with:

```
# Standard Python environment
pip install "harmonic-drive-twin[dev]"
pytest tests/ -v

# Isaac Sim 6 environment
isaacsim6-python -m pip install "harmonic-drive-twin[dev]"
isaacsim6-python -m pytest tests/ -v
```

All 26 tests pass in under $60\,\mathrm{s}$ on a modern workstation.

## 10 Troubleshooting

| Symptom | Likely cause |
|---|---|
| `num_dof = 0` after `art.initialize()` | Prim path conflict or anchor body is kinematic. Use `add_reference_to_stage(prim_path="/World/MyRobot")` and make the anchor body dynamic + FixedJoint. |
| `KeyError: 'DriveJoint'` | The DOF name does not match the USD joint prim name. Print `art.dof_names` to find the correct name. |
| Arm swings in wrong direction | Torque sign: check that positive effort pushes toward positive angle. Apply `T_cmd` directly (not `sign * T_cmd * sign`). |
| $\eta > 1$ or $\eta < 0$ | Pass `abs(T_out)` to `solve()`; the model expects a non-negative output torque. |
| `LD_PRELOAD` error on ARM64 | Add `os.environ.setdefault("LD_PRELOAD", "/lib/aarch64-linux-gnu/libgomp.so.1")` before any `isaacsim` import. |